

OpenShift Networking

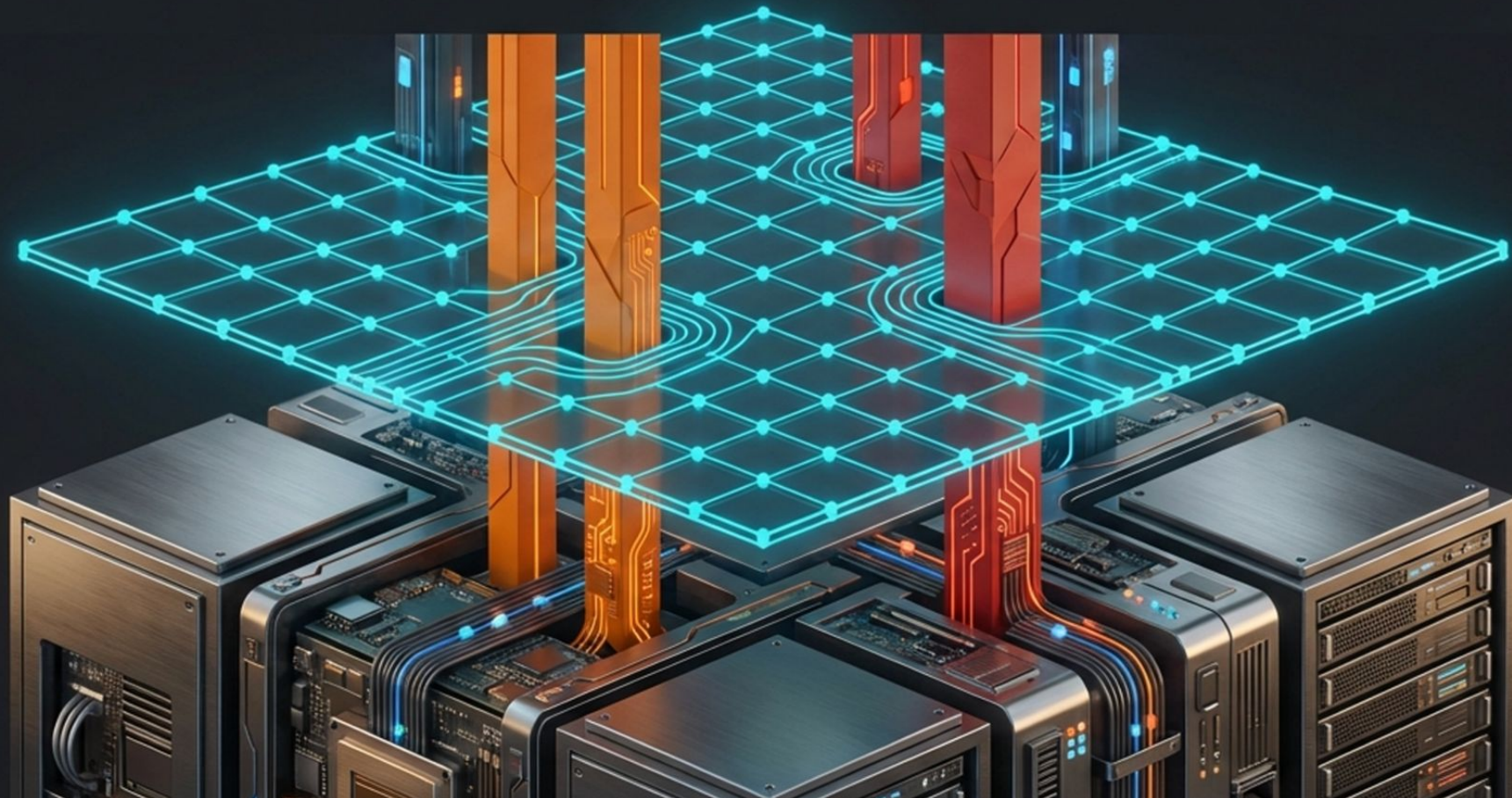
Default Network, Multus, and Network
Attachment Definition

Agenda

- ▶ Default OpenShift networking model
- ▶ Pod and Service networking
- ▶ Why Multus and secondary networks
- ▶ Multus CNI overview
- ▶ NetworkAttachmentDefinition (NAD)
- ▶ Using a secondary interface in a Pod
- ▶ Extension patterns and best practices

The Multi-Dimensional Datacenter

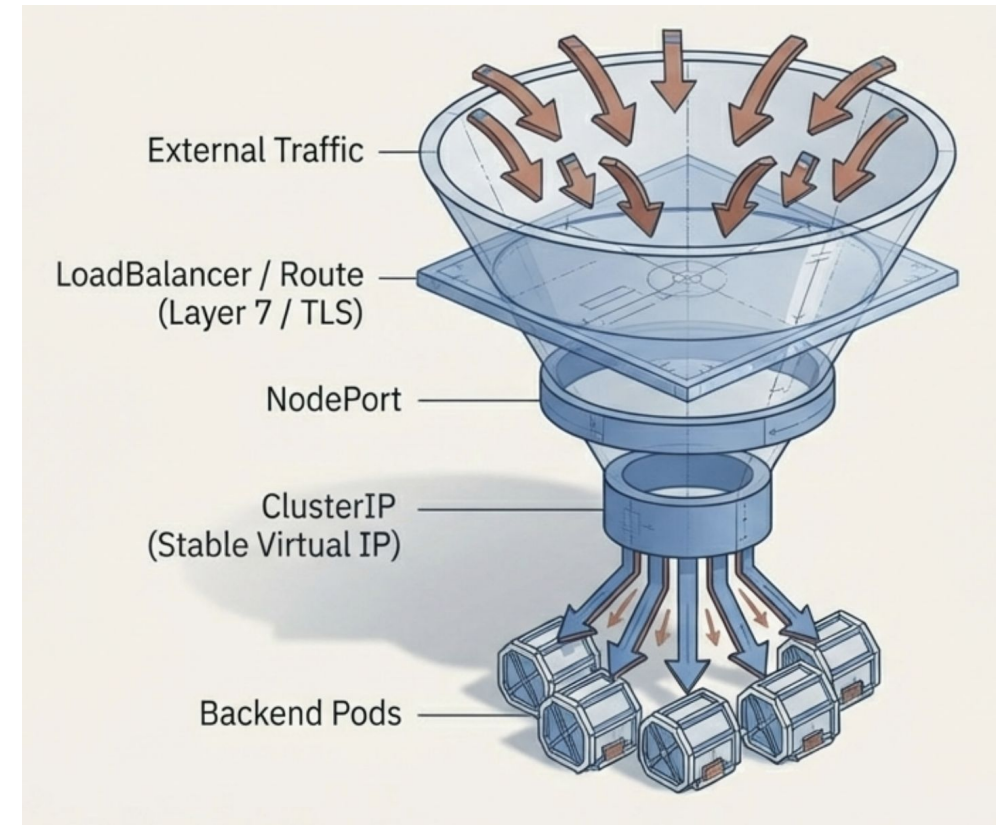
Transcending the Single-Interface Boundary with **OpenShift** and **Multus**



Default Cluster Network

Single primary interface by default

- ▶ OpenShift uses a cluster CNI (OVN-Kubernetes in current releases)
- ▶ Each Pod receives one pod network interface (eth0)
- ▶ Pod-to-Pod and Pod-to-Service traffic uses this overlay
- ▶ Ingress and Routes expose applications to the cluster edge
- ▶ NetworkPolicy defines allowed traffic between Pods

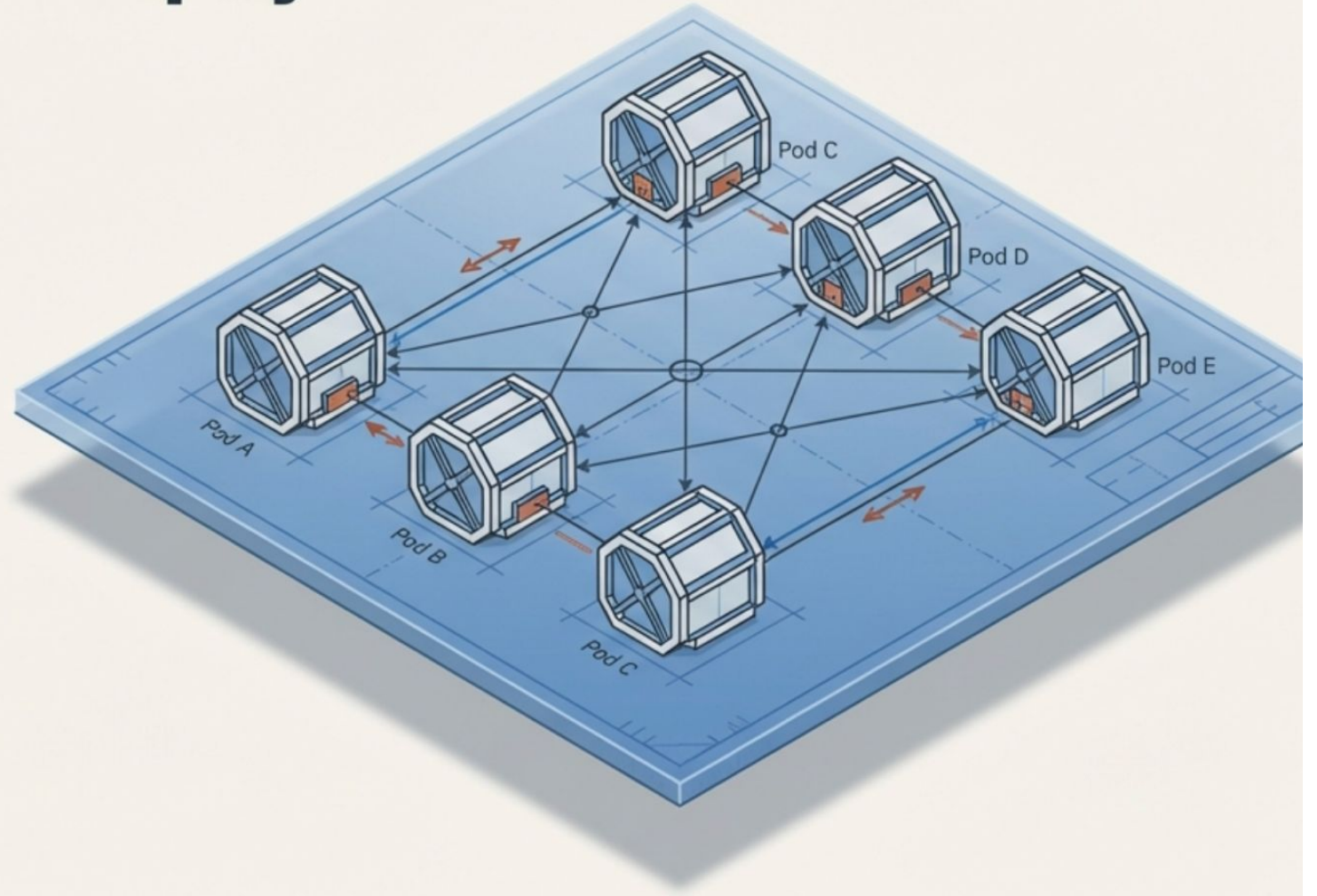


The Baseline: The Kubernetes Networking Philosophy

Flat cluster topology

Out-of-the-box pod-to-pod communication

Decoupled IP management

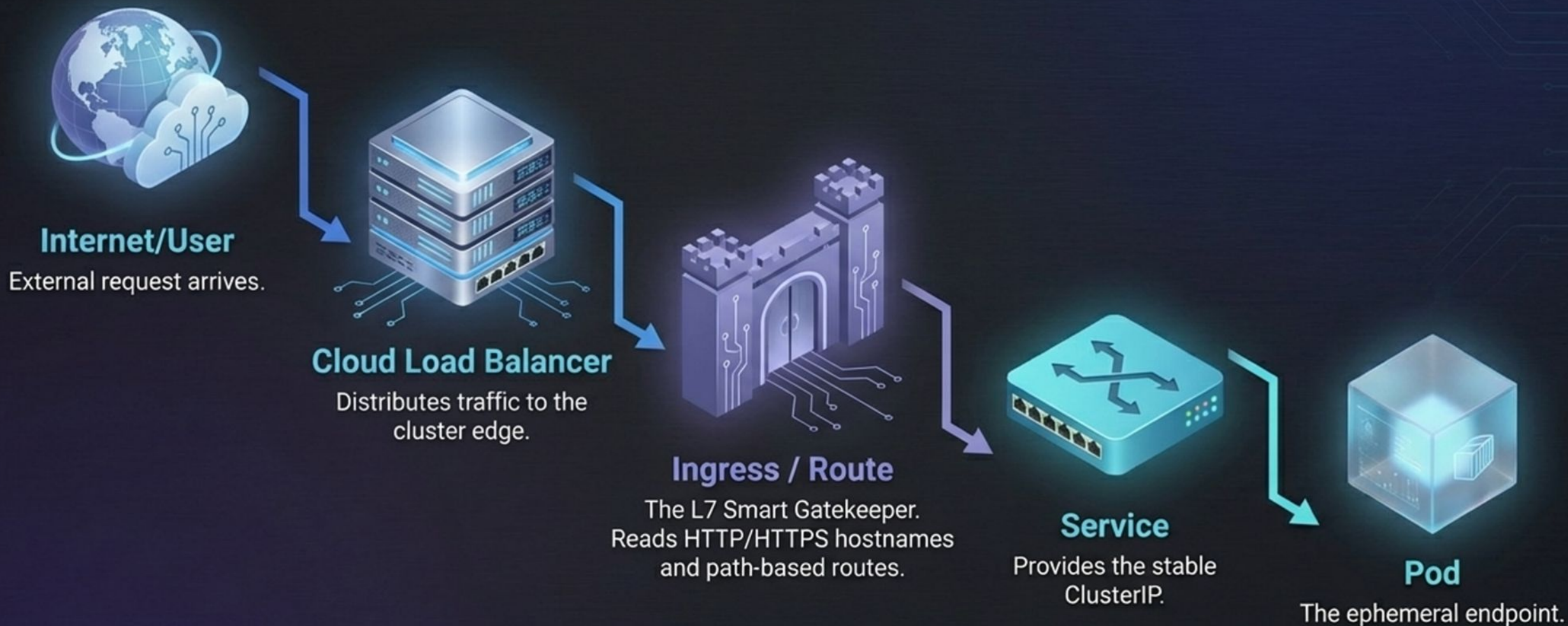


Services and DNS

Built-in abstractions

- ▶ Cluster DNS resolves Service names to ClusterIP
- ▶ kube-proxy or dataplane equivalent (OVN) implements Service routing
- ▶ LoadBalancer and Route types integrate with cloud or MetalLB
- ▶ Egress traffic typically uses NAT via the node
- ▶ NetworkPolicy is Kubernetes-native; applies to the default interface

The North-South Packet Journey



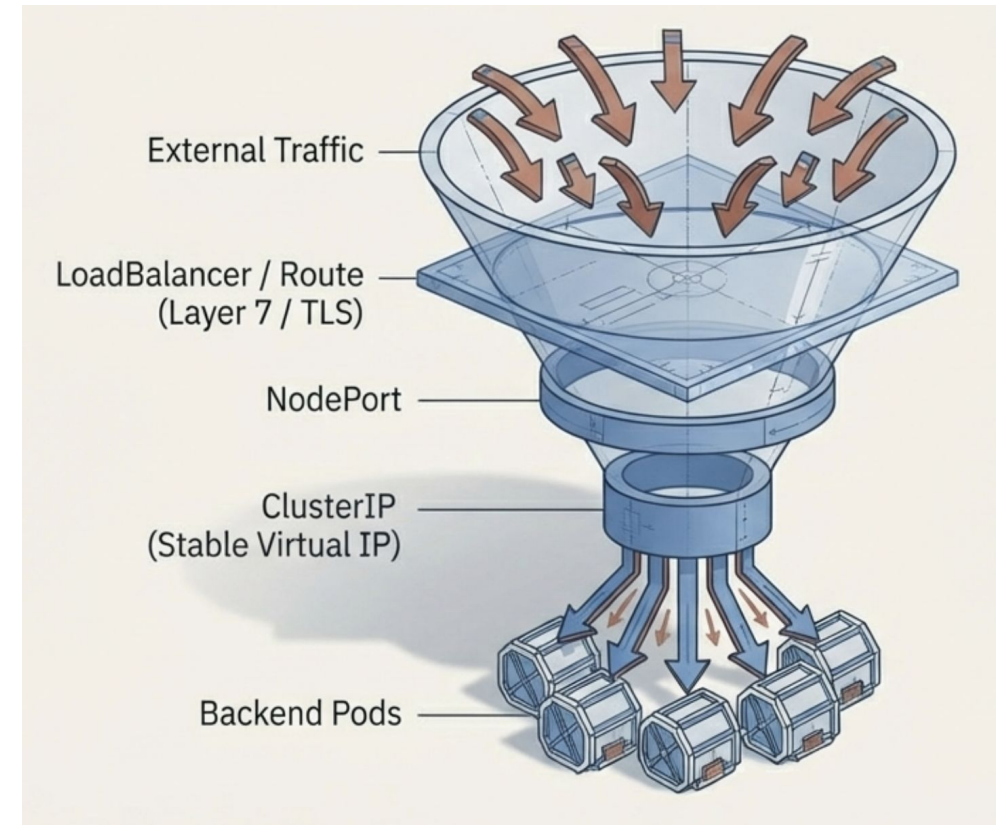
Traffic flows from external entrypoints through the Ingress/Route layer, which acts as a Layer 7 proxy. It reads the request path and directs it to the appropriate Service, which load-balances across the backing Pods.

Why Extend the Network

Limits of a Single Interface

Real-world requirements

- ▶ Attach Pods to VLANs or physical segments for isolation
- ▶ Separate data and control traffic for performance or compliance
- ▶ Connect to legacy networks without routing everything through the cluster
- ▶ SR-IOV or MACVLAN for low latency or hardware offload
- ▶ Multiple networks per Pod – not possible with default CNI alone



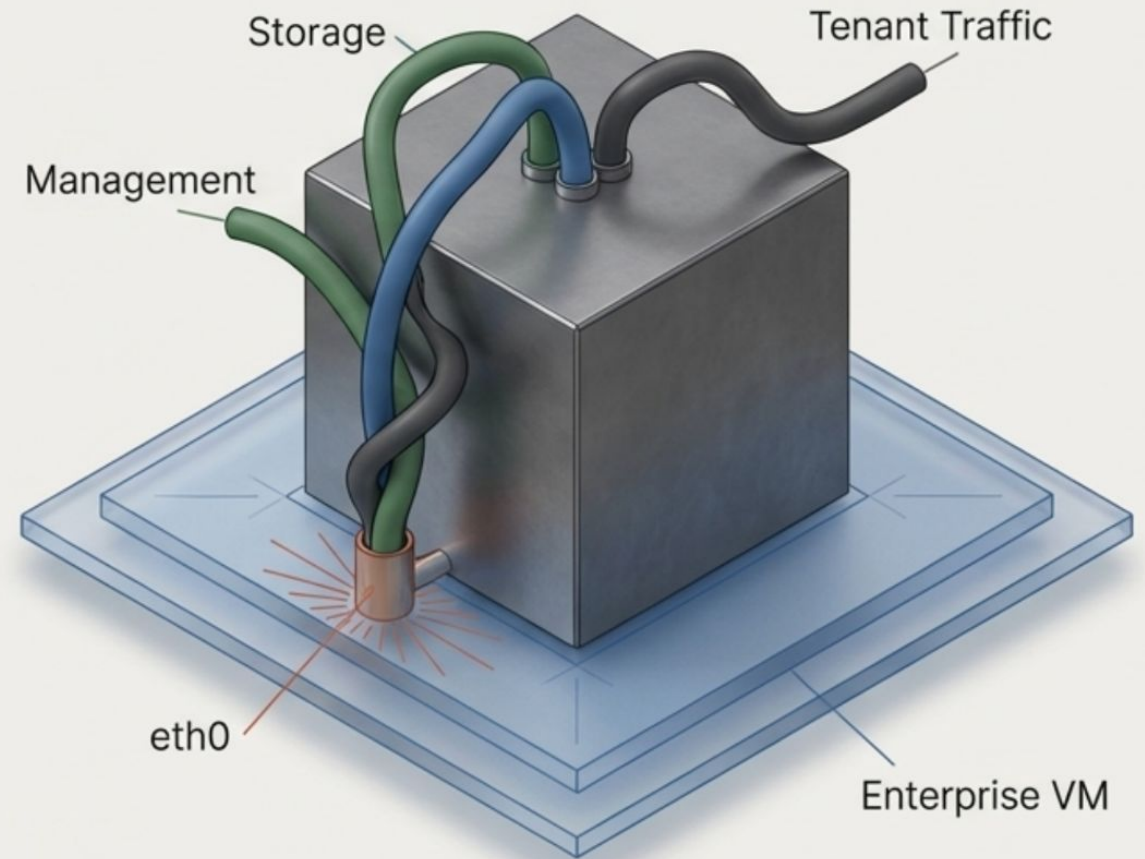
The Enterprise Constraint: The Single-Interface Bottleneck

The Limitation:

Kubernetes fundamentally assumes a 1-Pod-to-1-Interface (eth0) architecture.

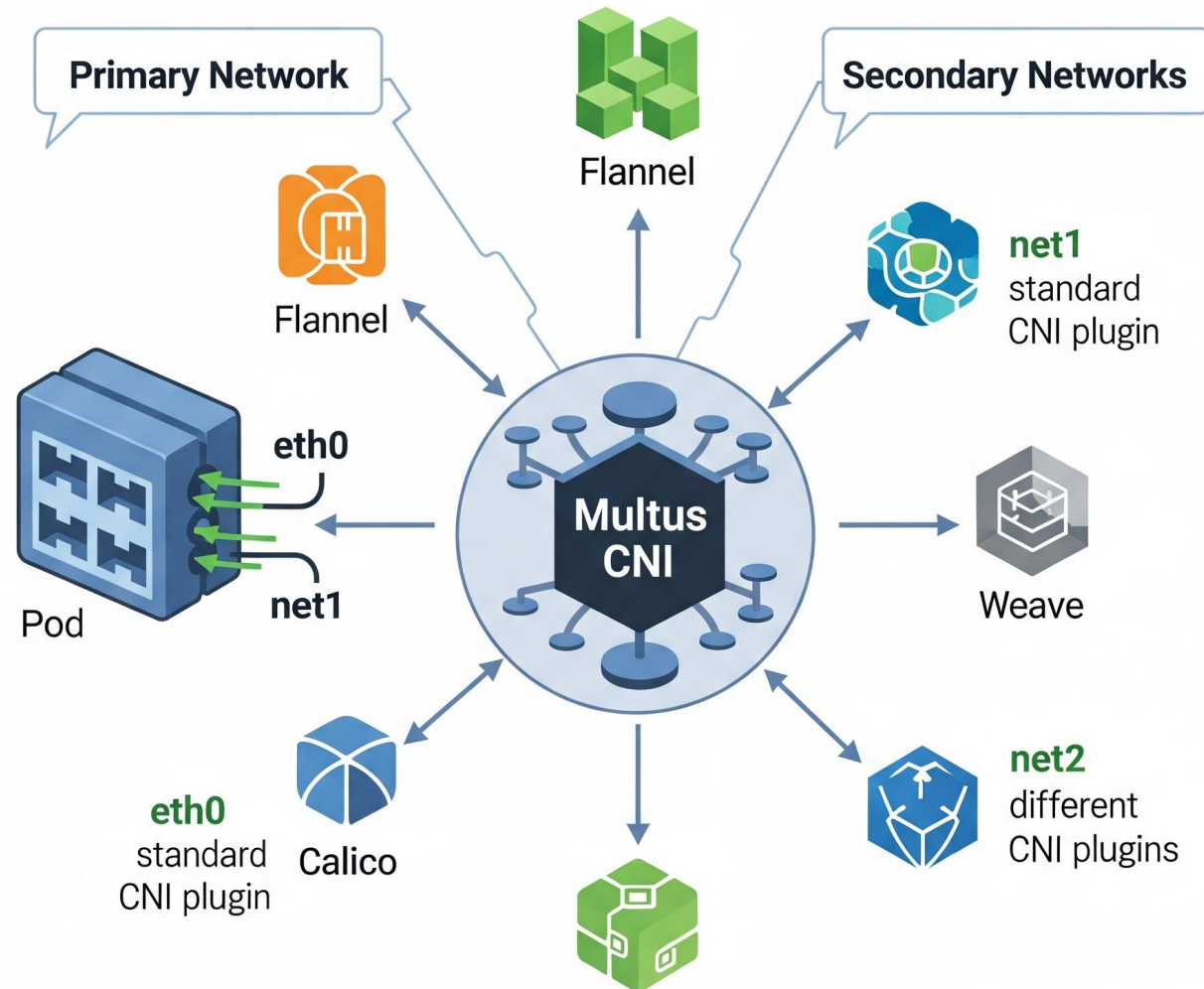
The Enterprise Reality:

VMs require dedicated storage networks, direct L2 access, and live migration paths.



Multus CNI: Multiple Network Interfaces for Kubernetes Pods

Multus



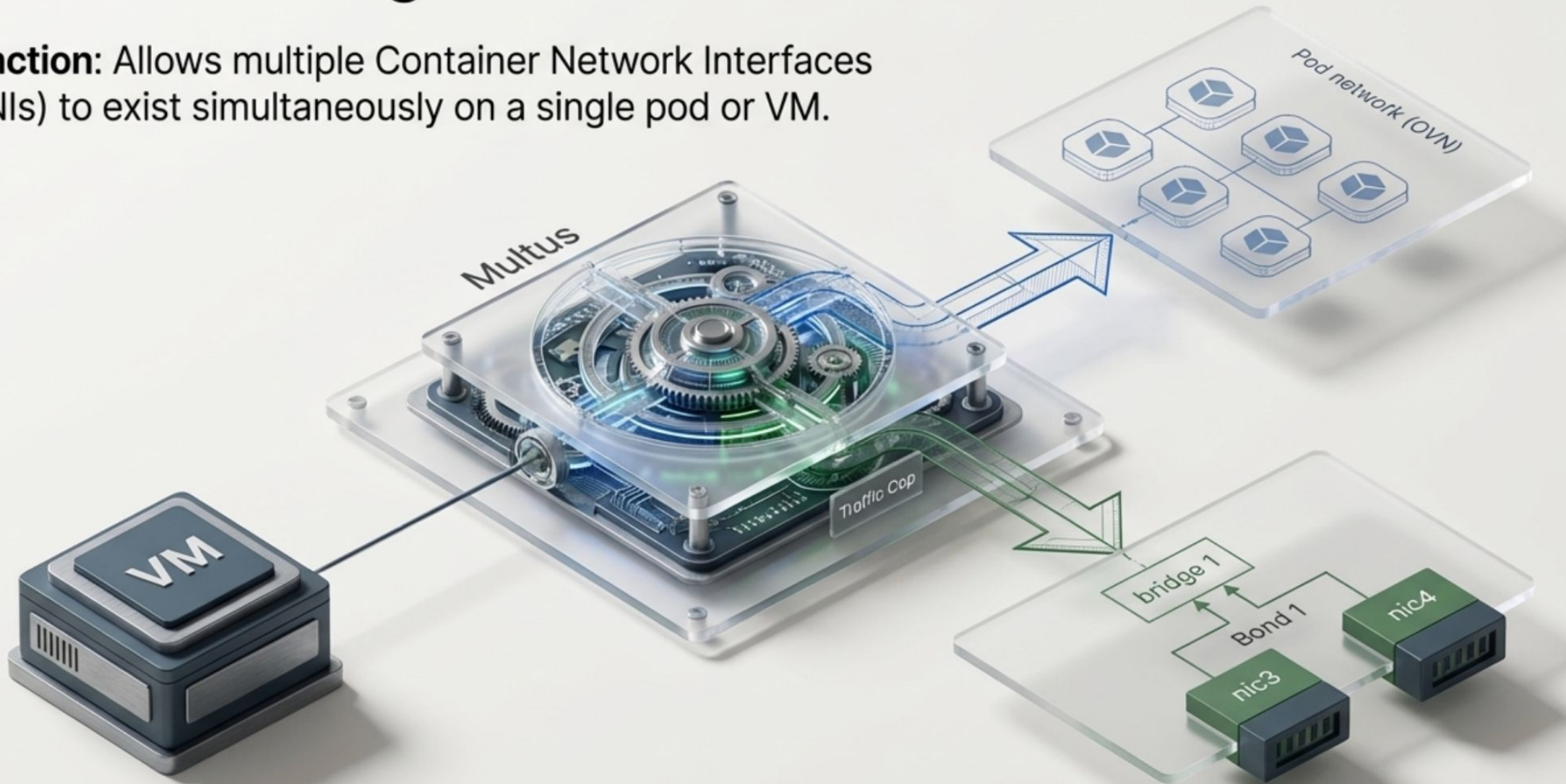
What Is Multus

Chaining additional CNIs

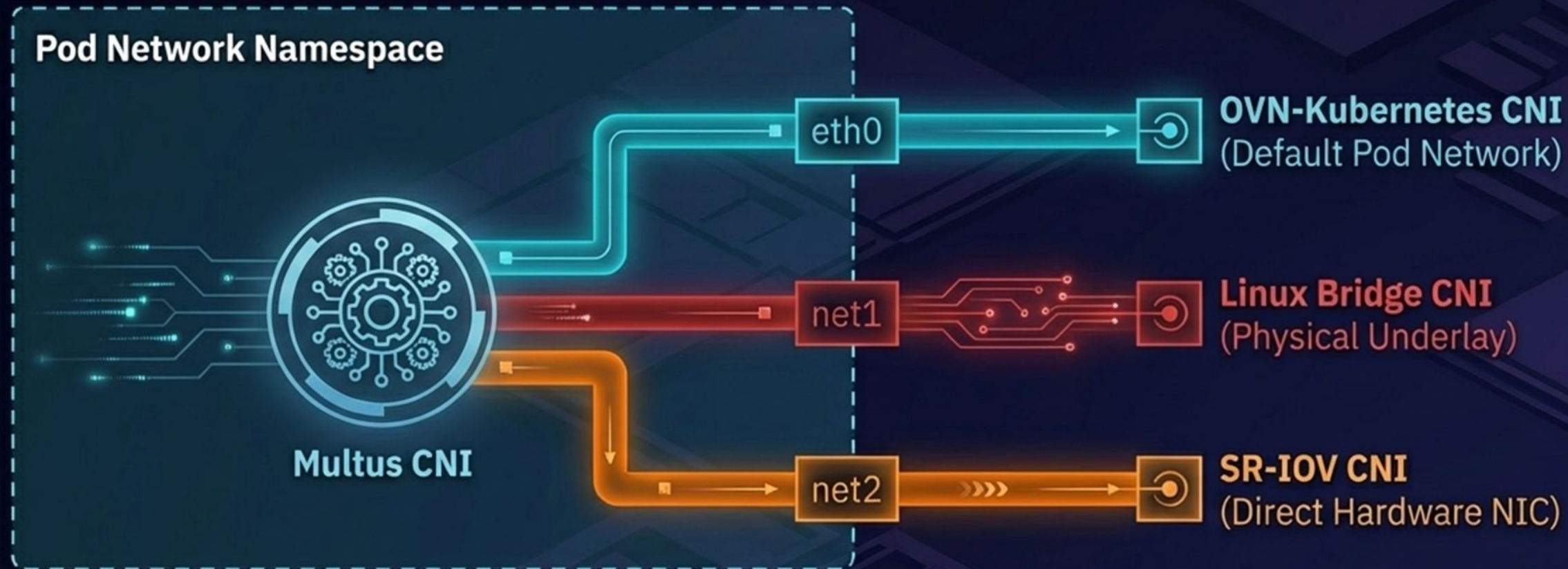
- ▶ Multus is a CNI that invokes other CNIs
- ▶ Keeps the default cluster network as primary (eth0)
- ▶ Adds secondary interfaces (eth1, net1, ...) per Pod spec
- ▶ Shipped and supported in OpenShift as a cluster component
- ▶ Works with bridge, macvlan, ipvlan, SR-IOV, and others

The Breakthrough: Multus as the Meta-CNI

Function: Allows multiple Container Network Interfaces (CNIs) to exist simultaneously on a single pod or VM.



Inside the Pod: The Meta-CNI Multiplexer



When a Pod is scheduled, the kubelet calls Multus. Multus then acts as a traffic director, invoking the default CNI for primary cluster connectivity, and invoking secondary CNIs to plumb additional interfaces (**net1**, **net2**) into the Pod namespace.

Multus in OpenShift

Day-2 model

- ▶ Cluster Network Operator manages Multus deployment
- ▶ Additional networks are defined in cluster-wide or namespace-scoped resources
- ▶ SR-IOV Operator can provision hardware-backed NICs for Pods
- ▶ Where applicable, MetalLB for LoadBalancer Services on bare metal
- ▶ Documentation: OpenShift Networking – Additional networks



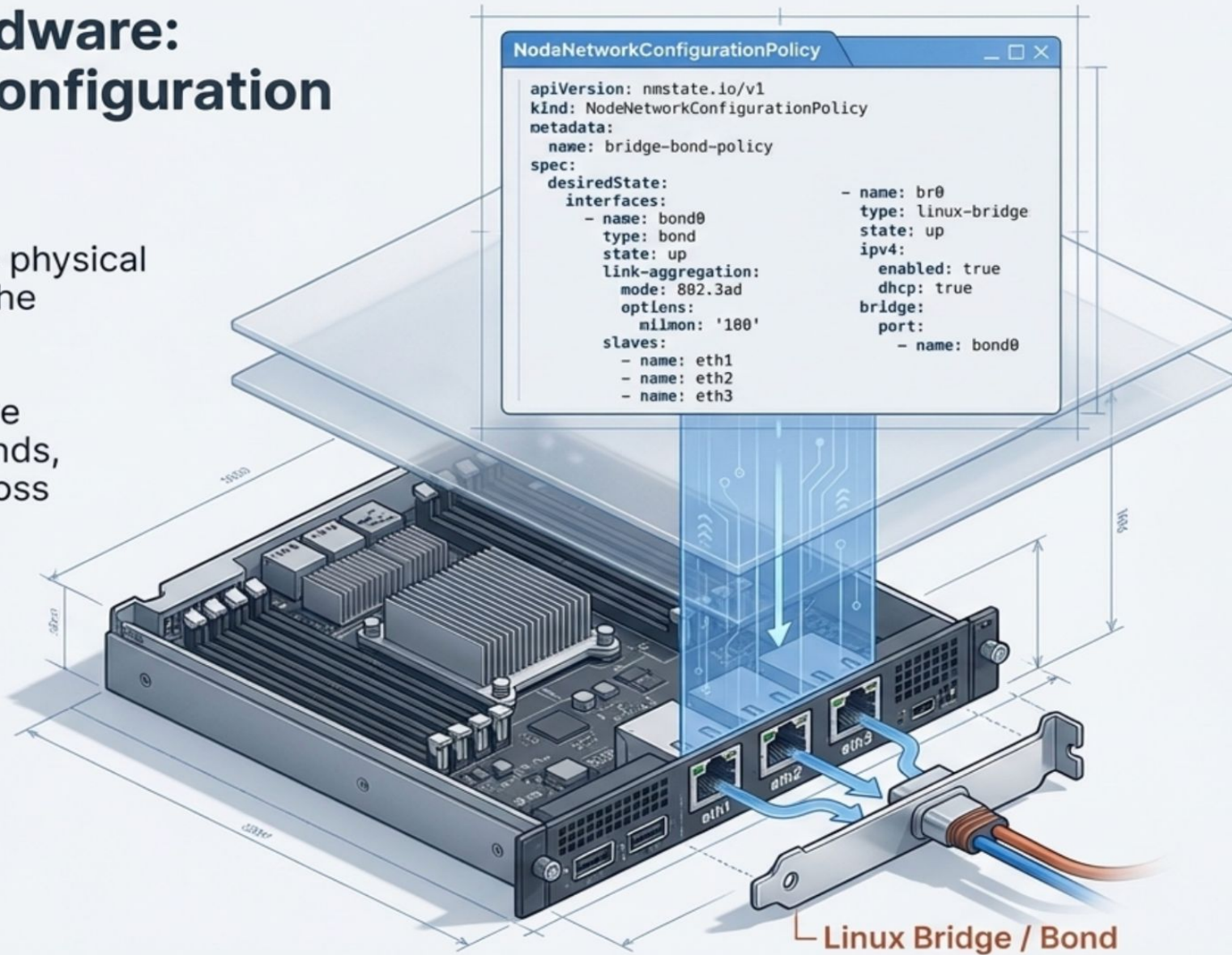
NetworkAttachmentDefinition (NAD)

CRD used by Multus

- ▶ NAD is a Custom Resource Definition (k8s.cni.cncf.io)
- ▶ Specifies a CNI configuration JSON as a string
- ▶ Names the logical network (referenced by Pods)
- ▶ Can be cluster-scoped or namespaced depending on design
- ▶ Admin creates NAD; developers reference it in Pod annotations

Defining the Hardware: Node Network Configuration Policy (NNCP)

- **Mechanism:** Declarative physical node networking using the nmstate project.
- **Capabilities:** Add/remove interfaces, configure bonds, set up Linux bridges across cluster nodes.



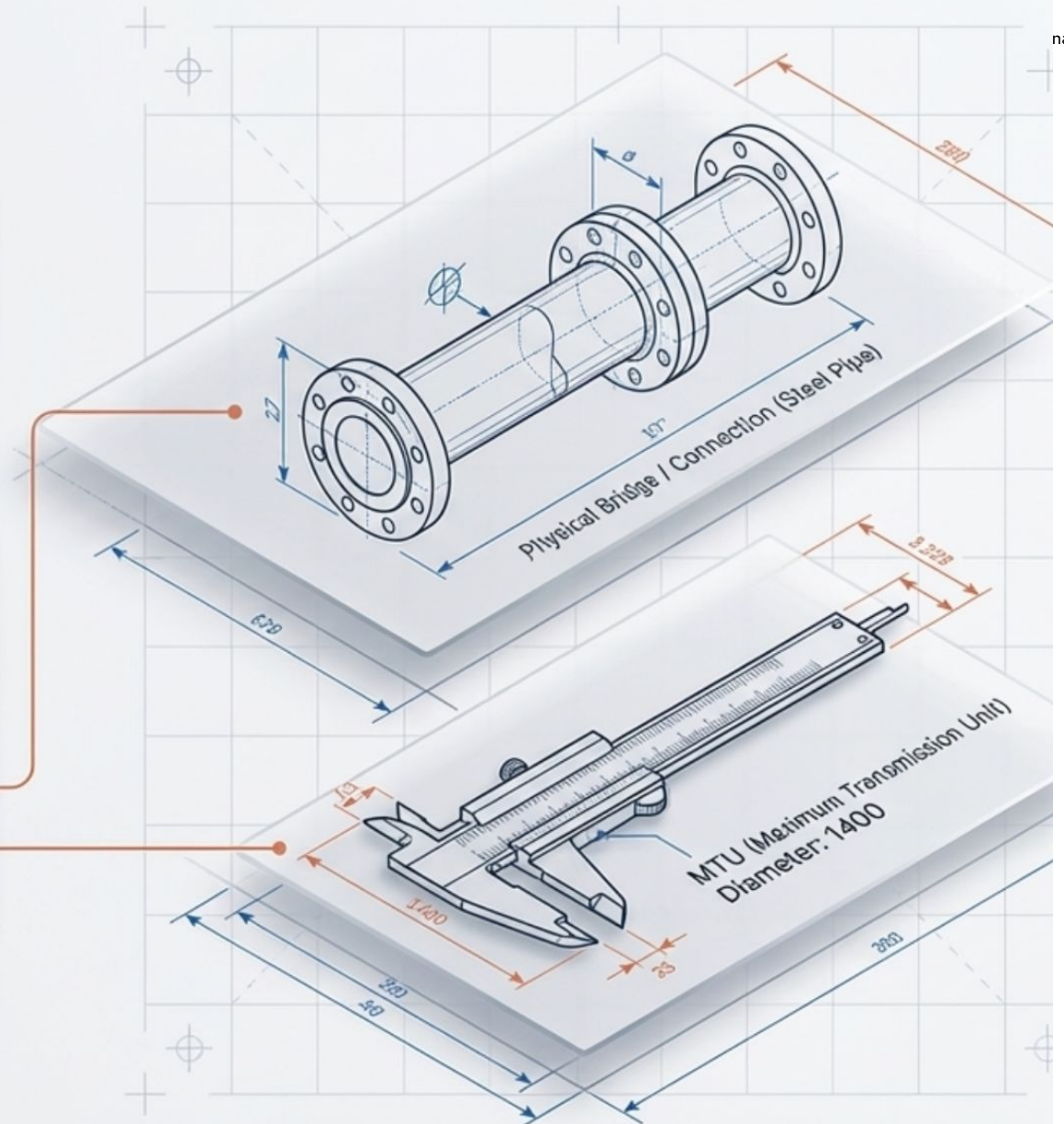
Defining the Attachment: The Anatomy of a NAD

The Multus Custom Resource Definition that binds workloads to configured networks.

Note: IPAM configuration in a NAD is not supported for virtual machines.

Recipe Card

```
kind: NetworkAttachmentDefinition
apiVersion: k8s.cni.cncf.io/v1
cniVersion: 0.3.1
type: bridge ]
mtu: 1400 ]
```

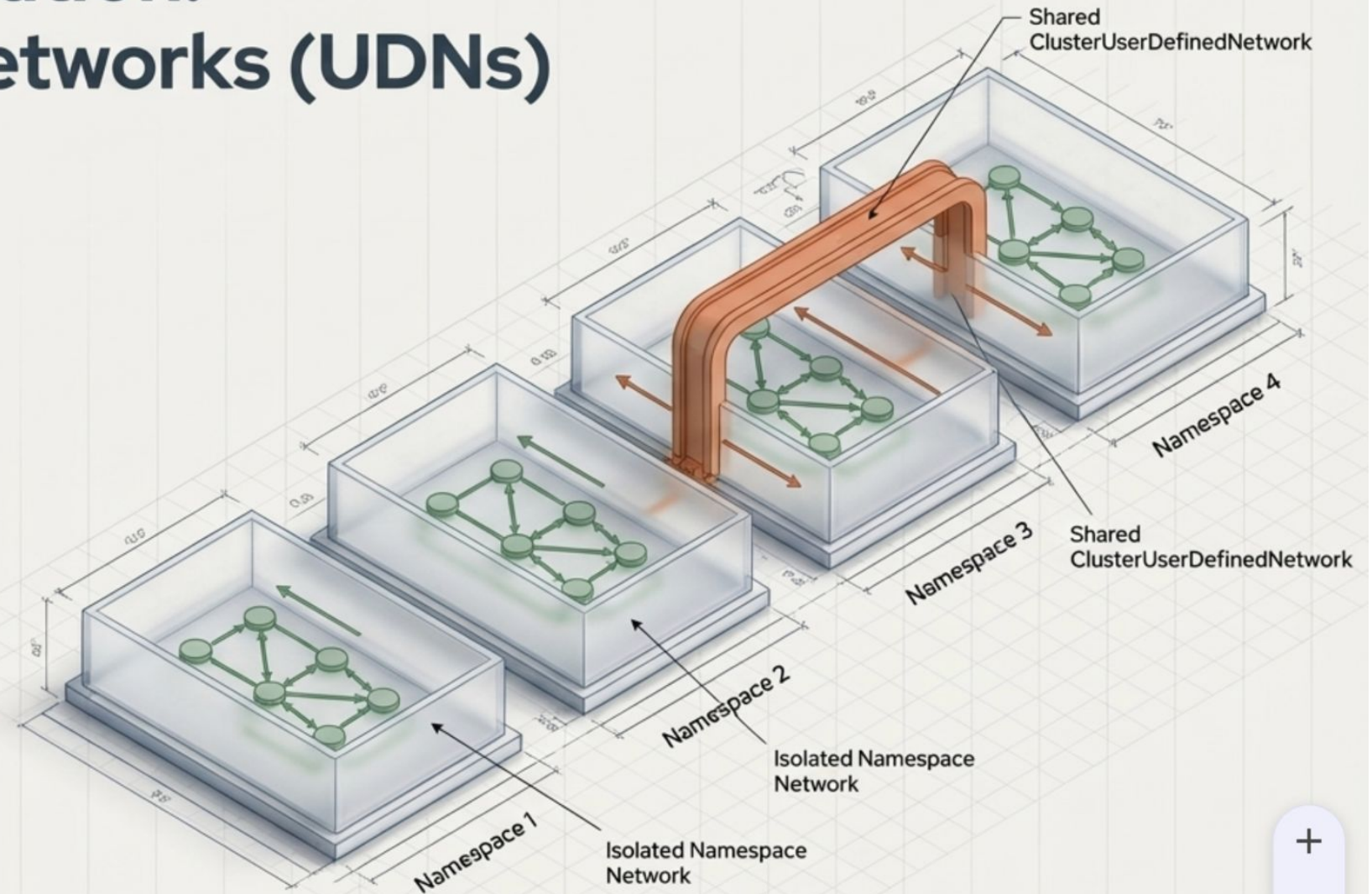


nator here

Namespace Isolation: User Defined Networks (UDNs)

Tenant Isolation:
Namespace-scoped CRD
creating isolated networks.

Shared Access:
ClusterUserDefinedNetwork
allows shared networks
across multiple specified
namespaces.



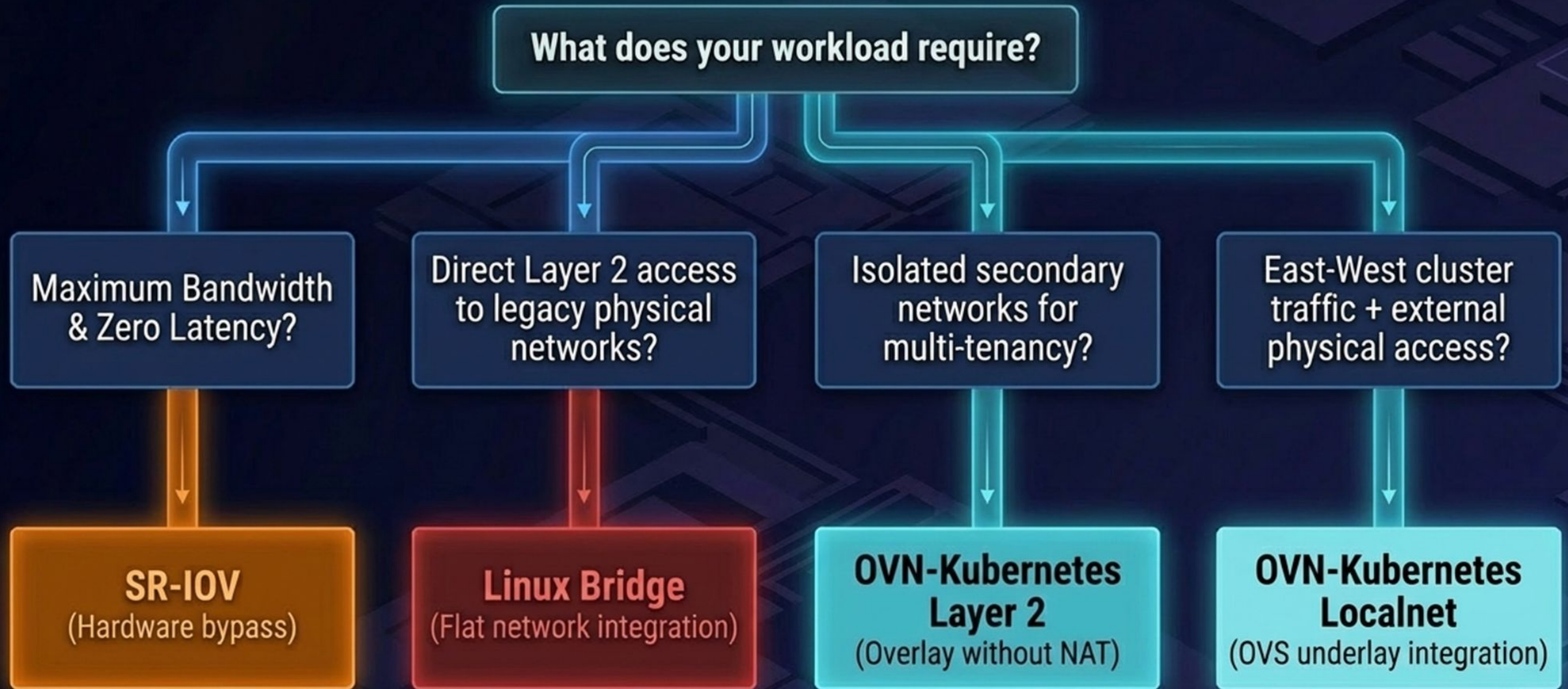
21

Version number here V00000

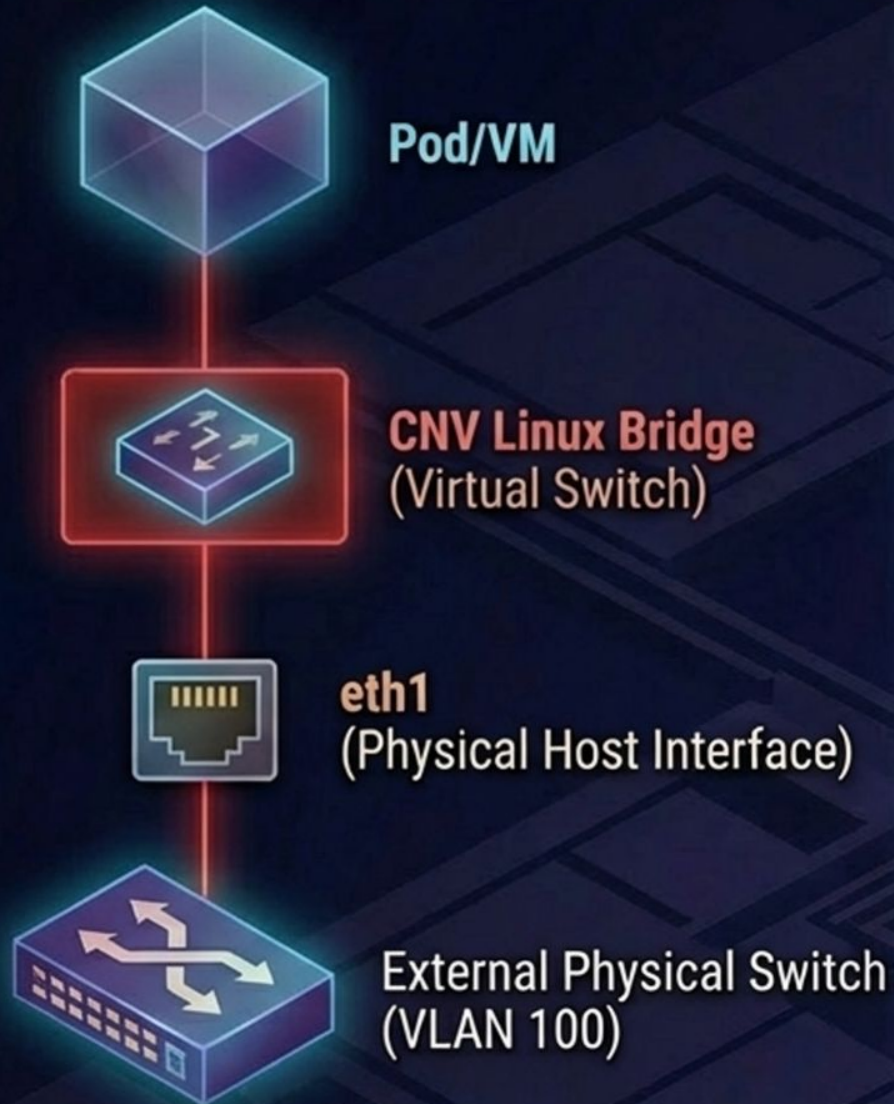


Extensions and Best Practices

Secondary Network Topologies: A Decision Matrix



Topology Deep Dive: Linux Bridge

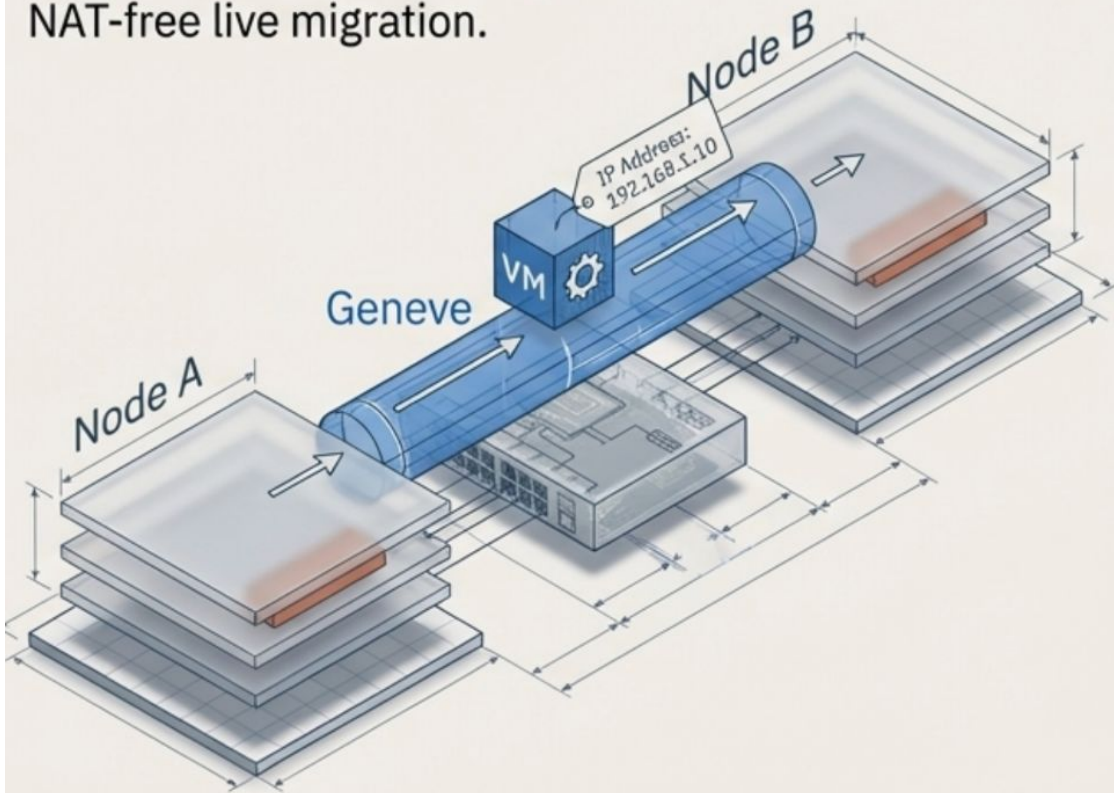


- **Mechanism:** Creates a virtual Layer-2 switch on the host node.
- **Integration:** Bridges a secondary Pod/VM interface directly to a physical network on the host.
- **Port Isolation:** Can be enabled to ensure VMs on the same VLAN operate in isolation from one another.
- **Use Case:** Ideal for OpenShift Virtualization VMs that must exist on the exact same broadcast domain as legacy bare-metal servers.

Extending the Underlay: L2 Geneve vs. Localnet

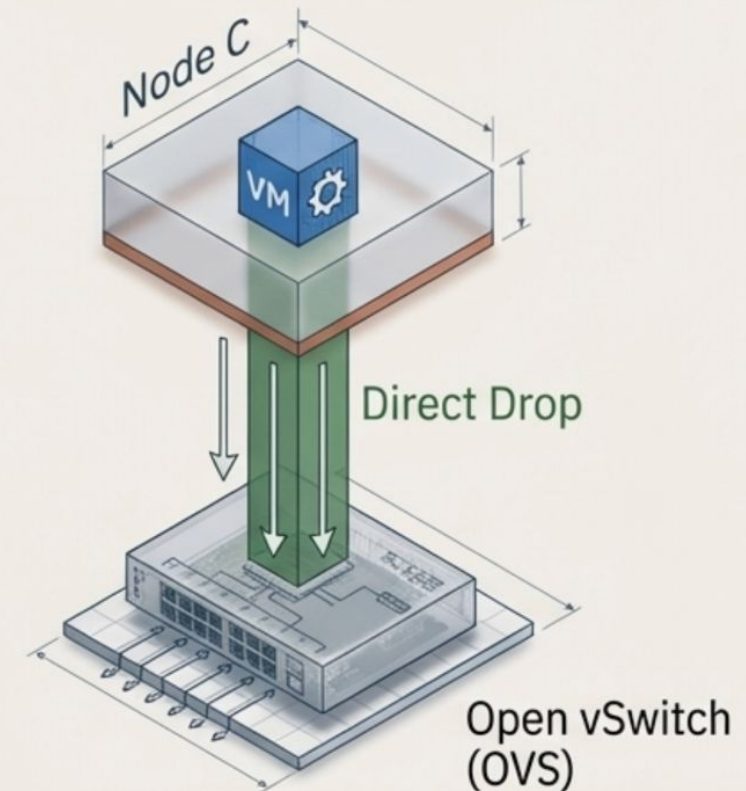
Layer 2 Overlay Topology

Geneve tunnel encapsulation.
Preserves persistent IPs for
NAT-free live migration.



Localnet Topology

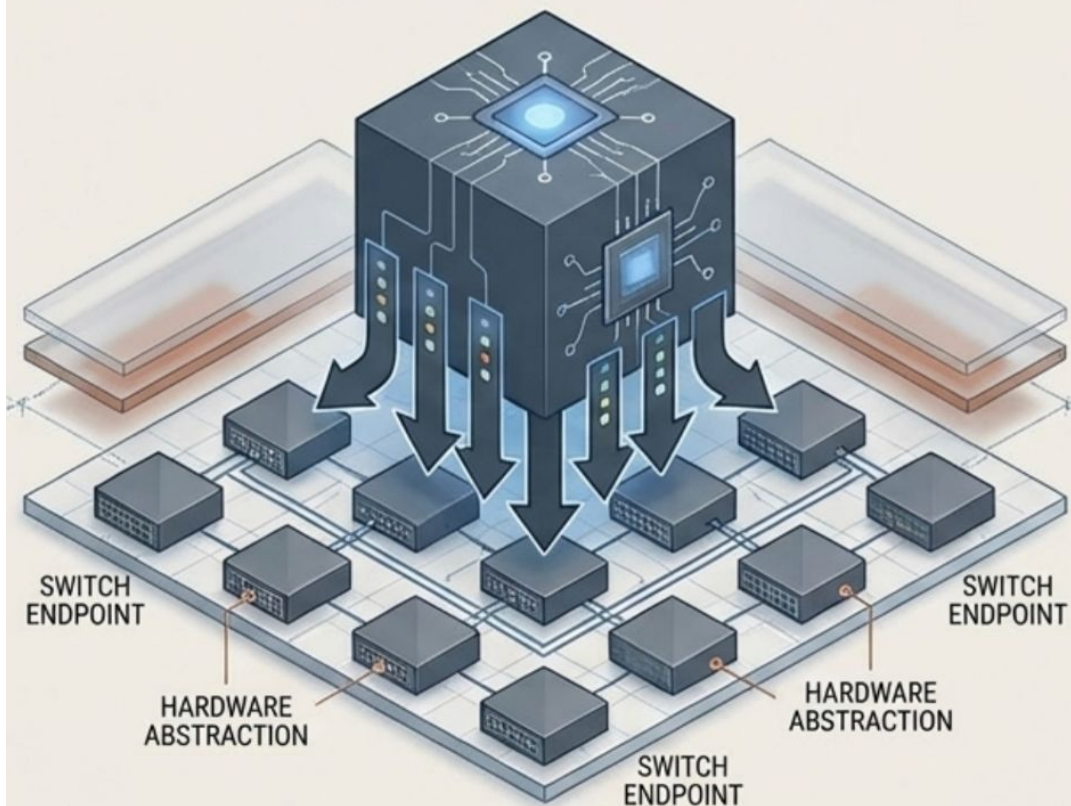
Direct drop to physical OVS layer.
Enables east-west and external access.



OVN vs NSX

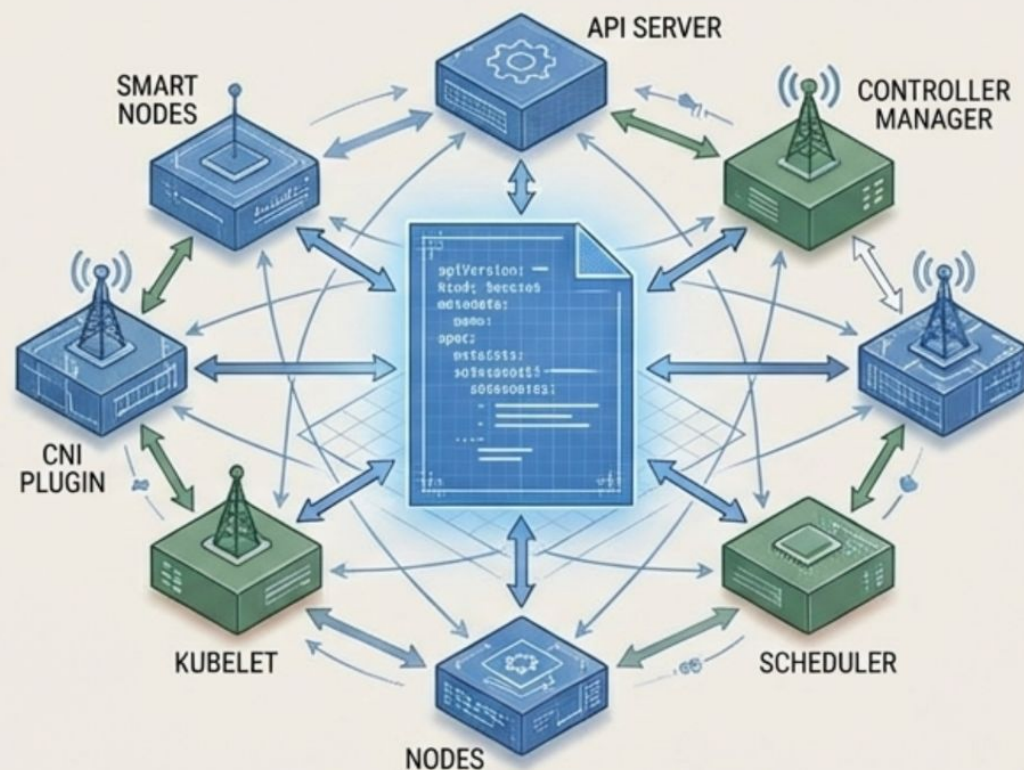
The Rosetta Stone: Bridging the Mindsets

VMware NSX Paradigm



Centralized controller, explicit switch configuration, opaque hardware abstraction.

OpenShift Paradigm



Declarative intent, distributed execution (CNIs), infrastructure-as-code CRDs.

Architecture Translation Matrix

nator here

VMware NSX Paradigm	OpenShift (OVN-K + Multus) Paradigm
NSX Overlay (Geneve)	OVN-Kubernetes default pod network
NSX VLAN-backed logical switches	Multus + Bridge/SR-IOV CNI
NSX Logical Ports	NetworkAttachmentDefinition (NAD)
ESXi vSwitch / Transport Nodes	NodeNetworkConfigurationPolicy (NNCP)
Storage Policy Based Management (SPBM)	Kubernetes Storage Classes

+

-

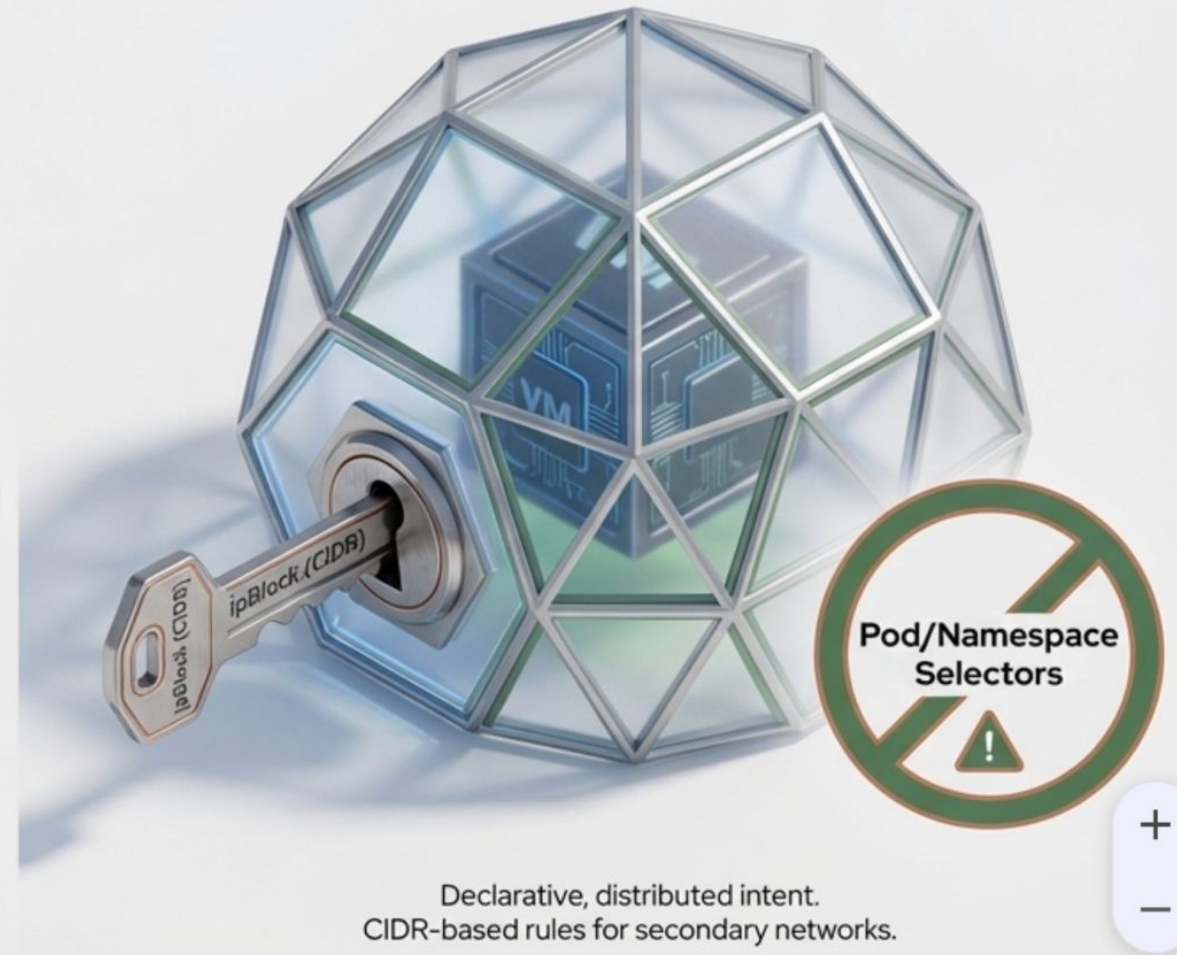
Policy & Security: Micro-segmentation Translated

VMware: NSX Distributed Firewall (DFW)



Centralized, rule-based enforcement.
Opaque, strict perimeter defense.

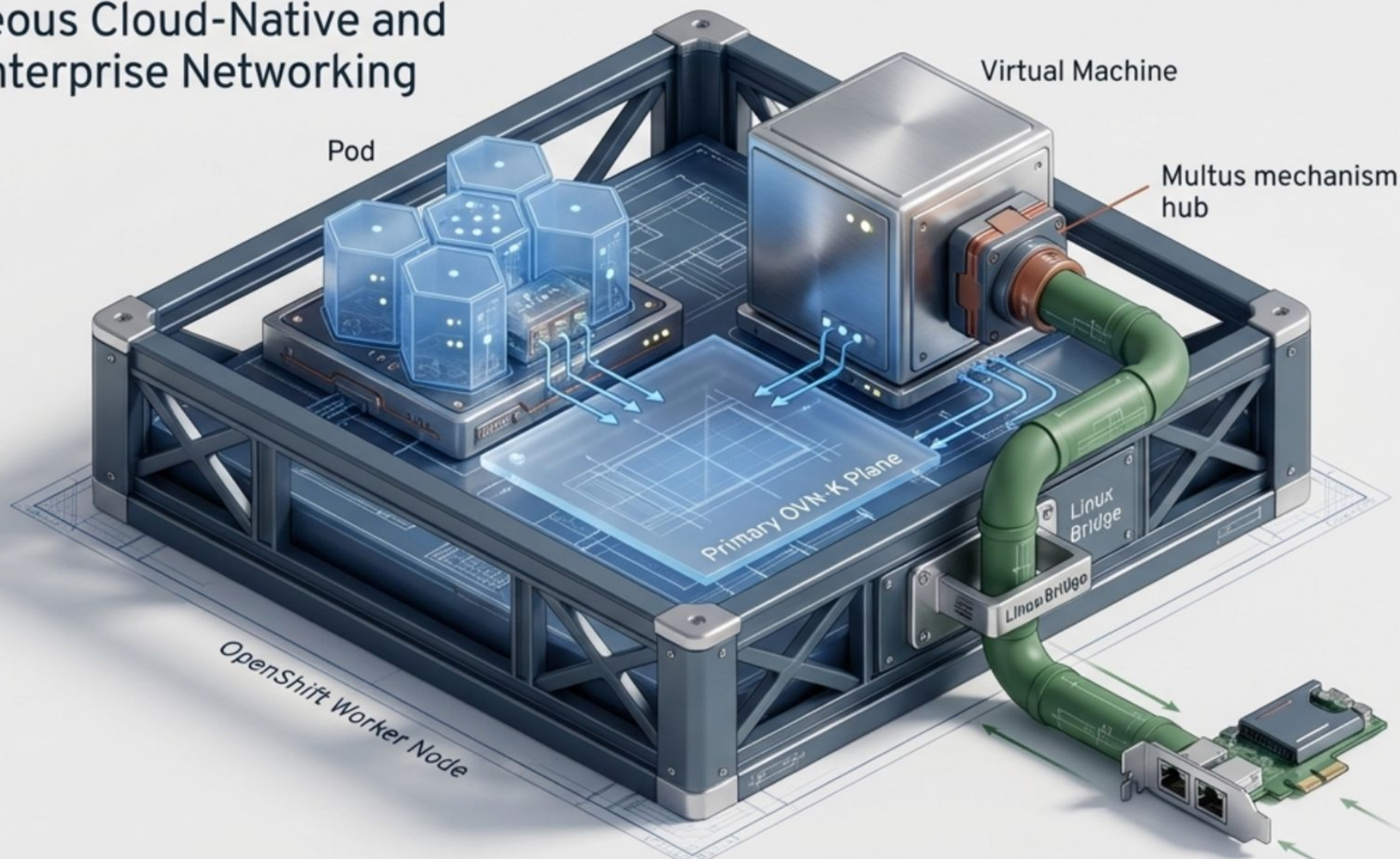
OpenShift: MultiNetworkPolicy API



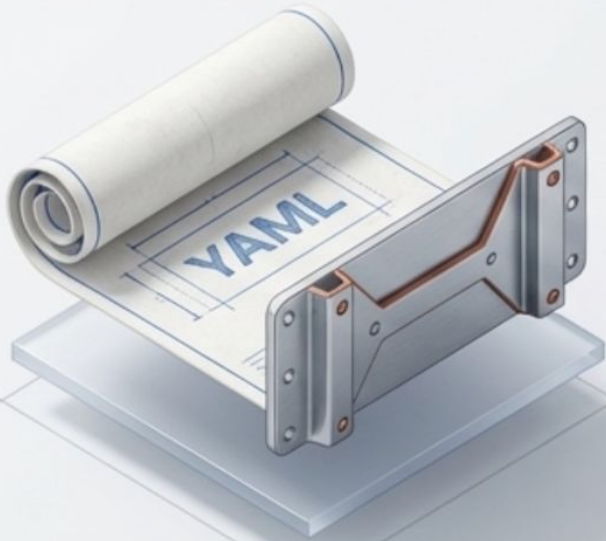
Declarative, distributed intent.
CIDR-based rules for secondary networks.

Synthesis: The Unified Master Node

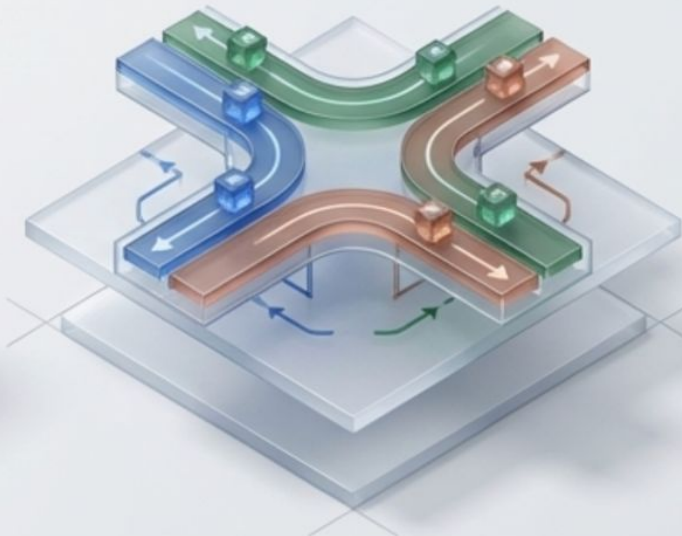
Simultaneous Cloud-Native and
Legacy Enterprise Networking



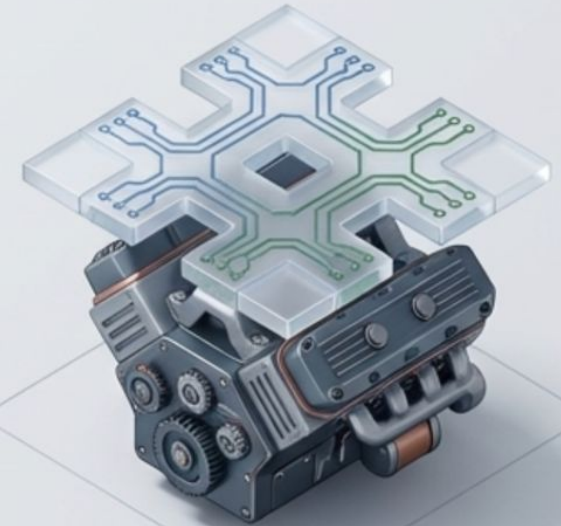
Key Takeaways for the Network Architect



1. The Network is Declarative:
Physical state is now code (NNCP/NAD).



2. Multus is the Enabler:
It unlocks L2, SR-IOV, and VLANs for legacy VMs.



3. OVN-Kubernetes is the Core:
Rely on it for seamless overlays and live migration via Geneve.

Thank You

Questions? See OpenShift Networking – Additional networks and Multus.



linkedin.com/company/red-hat



youtube.com/user/RedHatVideos



facebook.com/redhatinc



twitter.com/RedHat